

DVB-T

DVB-T is a standard for transmitting terrestrial digital video broadcast, which is used in the majority of Africa, Asia, Australia and Europe. It is possible to receive DVB-T using several different hardware setups, however this article will focus on DVB-T USB dongles based on the RTL2832U chipset (which are also very popular as cheap software defined radios using [RTL-SDR](#)).

Related articles

[DVB-S](#)[LIRC](#)[RTL-SDR](#)

1 Driver

The main driver in use is `dvb_usb_rtl28xxu`, and exists in the latest kernels. If it is not [loaded](#), do so manually:

```
# modprobe dvb_usb_rtl28xxu
```

You might also need to load `rtl2832` or `rtl2830`:

```
# modprobe rtl2830
# modprobe rtl2832
```

Note

If you have [RTL-SDR](#) installed, note that it conflicts with this driver, and therefore blacklists it. Make sure to remove any necessary blacklists before loading the driver. The default location for the blacklist file is in `/etc/modprobe.d/rtlsdr.conf`.

After plugging the device in, the output of [dmesg](#) should show something like this:

```
[ 4009.326338] usb 7-5: new high-speed USB device number 4 using ehci-pci
[ 4009.466712] usb 7-5: dvb_usb_v2: found a 'Realtek RTL2832U reference design' in warm state
[ 4009.531594] usb 7-5: dvb_usb_v2: will pass the complete MPEG2 transport stream to the software demuxer
[ 4009.531613] DVB: registering new adapter (Realtek RTL2832U reference design)
[ 4009.534554] usb 7-5: DVB: registering adapter 0 frontend 0 (Realtek RTL2832 (DVB-T))...
[ 4009.534627] r820t 4-001a: creating new instance
[ 4009.546177] r820t 4-001a: Rafael Micro r820t successfully identified
[ 4009.552681] Registered IR keymap rc-empty
[ 4009.552783] input: Realtek RTL2832U reference design as /devices/pci0000:00/0000:00:1d.7/usb7/7-5/rc/rc1/input20
[ 4009.552854] rc1: Realtek RTL2832U reference design as /devices/pci0000:00/0000:00:1d.7/usb7/7-5/rc/rc1
[ 4009.553275] input: MCE IR Keyboard/Mouse (dvb_usb_rtl28xxu) as /devices/virtual/input/input21
[ 4009.554466] rc rc1: lirc_dev: driver ir-lirc-codec (dvb_usb_rtl28xxu) registered at minor = 0
[ 4009.554474] usb 7-5: dvb_usb_v2: schedule remote query interval to 400 msecs
[ 4009.565930] usb 7-5: dvb_usb_v2: 'Realtek RTL2832U reference design' successfully initialized and connected
```

Note

in this case we see that the dongle has a R820T tuner, but there are several other popular tuners that you might run into. Also note the IR sensor device that was recognized that, properly configured, can be used with the device remote control. See [LIRC](#) for more information.

Additionally, you should now see the adapter device under `/dev/dvb/adapter0`. Some cards need additional firmwares that are not distributed for various reasons. Usually you will find an explicit message about that from *dmesg*. Look for the name of the file(s) you see with your favorite search engine, and once you have them, put the required firmware(s) in `/usr/lib/firmware`. Possibly a package might exist in the AUR.

2 Utilities

Various DVB utilities can be found in the [linuxtv-dvb-apps](https://aur.archlinux.org/packages/linuxtv-dvb-apps/) (<https://aur.archlinux.org/packages/linuxtv-dvb-apps/>)^{AUR} package.

2.1 Scanning

[w_scan_cpp](https://aur.archlinux.org/packages/w_scan_cpp/) (https://aur.archlinux.org/packages/w_scan_cpp/)^{AUR} allows for automatic scanning of channels without configuration. Install it then issue:

```
# w_scan_cpp -ft -c country_code --output-mplayer > ~/channels.conf
```

If you do not know your country code, enter the following to get a list of codes.

```
# w_scan_cpp -c "?"
```

The application now provides help. Use `w_scan_cpp --help` for more information.

More advanced scanning options can be found under [DVB-S#Scanning channels](#).

When `w_scan_cpp` fails to find all expected channels you could try [w_scan2](https://aur.archlinux.org/packages/w_scan2/) (https://aur.archlinux.org/packages/w_scan2/)^{AUR}. It is a fork of the original `w_scan` and can be found on [GitHub](https://github.com/stefantalpalaru/w_scan2) (https://github.com/stefantalpalaru/w_scan2).

3 Clients

See also [how to disable screensaver when playing video/TV](#) by using configuration files or use `xset` command before and after player starts to enable/disable it. If you have installed [xscreensaver](https://archlinux.org/packages/?name=xscreensaver) (<https://archlinux.org/packages/?name=xscreensaver>) then you will need to use `xscreensaver-command` instead of `xset` to activate/deactivate screensaver from command line.

3.1 Kaffeine

Kaffeine **does not work** (https://bugs.kde.org/show_bug.cgi?id=397594) on [Wayland](#). On [X11](#), DVB-T works out-of-the box in [Kaffeine](#), including management of multiple DVB-T devices, channel tuning, channel selection, EPG and recording. No external playlist generation is needed. Multiple DVB-T devices can be used at once (e.g. for recording from a multiplex while watching another one). Many single-tuner DVB-T devices can even provide two different TV channels, as long as they share the same multiplex; this feature is also readily available in Kaffeine.

3.2 Smplayer

[Smplayer](#) Can play DVB-T with [tvheadend](#)

3.3 VLC

The simplest way to watch DVB-T channels with [VLC](#) is to first generate a playlist:

```
$ w_scan_cpp -ft -c country_code -L > dvb.xspf
$ vlc dvb.xspf
```

You can also specify the frequency and programs by hand. This can be done using:

```
$ vlc dvb://frequency=543000000
```

where the frequency is set in Hz, and should match the base frequency for the transmissions in your area. You can also explicitly specify which demodulation you would like to use, so instead of `dvb` you can use `dvb-t`, `dvb-t2`, etc.

VLC also accepts various command line arguments, for example if you want to tune into a different program:

```
$ vlc dvb://frequency=543000000 :program=3
```

If some [DVB-T streams do not work](#), [install aribb24](#) (<https://archlinux.org/packages/?name=aribb24>).

3.4 MPlayer / mpv

For DVB streaming, [MPlayer](#) (or [mpv](#)) requires a channels configuration file at `~/.mplayer/channels.conf`. Follow [#Scanning](#) for instructions on how to generate it, but make sure to use the `-M` flag to generate the proper format for MPlayer, if you are using `w_scan_cpp`:

```
$ w_scan_cpp -ft -c country_code -M > ~/.mplayer/channels.conf
```

For `mpv`, remove the `-M` flag to use the newer VDR file format that supports specifying the delivery system (DVB-T or DVB-T2) per channel:

```
$ w_scan_cpp -ft -c country_code > ~/.config/mpv/channels.conf.ter
```

Try the configuration with `mplayer dvb://`, which should start to play the first channel. If it does not, you might need to use `-demuxer lavf` or `-demuxer mpegts` in order to properly receive the stream.

If the configuration works, you can simply run:

```
$ mplayer dvb://"STREAM NAME"
```

with a valid `STREAM NAME` from the channels configuration file.

Note

MPlayer cannot handle and play channels from a command line that contains some of Linux special symbols like `/` in their names, but you can manually rename them by editing `~/.mplayer/channels.conf`.

3.4.1 Channel selector

Here is a [lsttv](https://aur.archlinux.org/packages/lsttv/) (<https://aur.archlinux.org/packages/lsttv/>)^{AUR} script that will show a numbered list of channels by reading data from a `~/.mplayer/channels.conf` file. You will be able to watch a channel by using a number associated to it by the script instead of having to type the whole channel name on the command line, e.g. `lsttv 3`. The channel number associated by the script equals to the line number with tuning configuration for it. The script disables display power saving and a screen saver before starting *mplayer* and enables both again after you close it to disable screensaver management in this script remove `xset ...`; before and after [MPlayer](#).

```
/usr/local/bin/lsttv
```

```
#!/bin/bash
if [ "$1" ];then
CC='^[0-9]+$';
if ! [[ "$@" =~ $CC ]];then echo Is not a channel number!;
else
##
awk -F':' -v AA="$1" '{ZZ++;
if(AA == ZZ)system("xset -dpms s off;mplayer dvb://"\"$1\"";xset +dpms s on")}'
END{if(AA > ZZ)printf "The highest channel number is: "ZZ"\n"}' "$HOME/.mplayer/channels.conf"
##
fi;
else
awk -F':' '{ZZ++; printf ZZ " | " $1 "\n"}' "$HOME/.mplayer/channels.conf"
fi;
```

Note

- It is assumed that the `channels.conf` file has been created with:
`w_scan_cpp -ft -c country_code -C UTF-8 -M -E 0 -O 0 > ~/.mplayer/channels.conf`
- If the list of channels is too long then you can use something like `lsttv | less` and search for channels name by pressing `/` and writing its name. When found press `q` for exiting of [less\(1\)](https://man.archlinux.org/man/less.1) (<https://man.archlinux.org/man/less.1>) and use the channel associated number with [lsttv](https://aur.archlinux.org/packages/lsttv/) (<https://aur.archlinux.org/packages/lsttv/>)^{AUR}.
- If you have a problem with playing of video see [Arch Linux Forum](https://bbs.archlinux.org/viewtopic.php?id=57650) (<https://bbs.archlinux.org/viewtopic.php?id=57650>).

Warning

If there is more than one channel with the same name, *mplayer* will play only the closest one in the list.

3.5 ffmpeg

FFmpeg can take DVB-T MPEG streams as input, but requires `tzap` (in [linuxtv-dvb-apps](https://aur.archlinux.org/packages/linuxtv-dvb-apps/) (<https://aur.archlinux.org/packages/linuxtv-dvb-apps/>)^{AUR}) to do so.

Note

This might not necessarily be the case, if a better method is known, please update.

First, generate a tzap-compatible `channels.conf` file, using `w_scan_cpp` :

```
$ w_scan_cpp -ft -A1 -X > ~/.tzap/channels.conf
```

Then, you can run:

```
$ tzap -r "CHANNEL NAME"
```

which, if setup correctly should yield an output similar to:

```
using '/dev/dvb/adapter0/frontend0' and '/dev/dvb/adapter0/demux0'
reading channels from file '/home/user/.tzap/channels.conf'
Version: 5.10    FE_CAN { DVB-T }
tuning to 506000000 Hz
video pid 0x0a21, audio pid 0x0a22
status 00 | signal 0000 | snr 0000 | ber 0000ffff | unc 00007fbd |
status 1f | signal 0000 | snr 0126 | ber 00000000 | unc 00007fbd | FE_HAS_LOCK
status 1f | signal 0000 | snr 0129 | ber 0000000f | unc 00007fbd | FE_HAS_LOCK
status 1f | signal 0000 | snr 0120 | ber 00000003 | unc 00007fbd | FE_HAS_LOCK
status 1f | signal 0000 | snr 0125 | ber 00000011 | unc 00007fbd | FE_HAS_LOCK
# ....
```

More information on `tzap` is available on the [zap wiki page \(https://www.linuxtv.org/wiki/index.php/Zap\)](https://www.linuxtv.org/wiki/index.php/Zap).

Once `tzap` is encoding the stream, `/dev/dvb/adapter0/dvr0` should be available to `ffmpeg` (or any other program).

A simple command to stream a program, without additional encoding might look like so:

```
$ ffmpeg -f mpegts -i /dev/dvb/adapter0/dvr0 out.mp4
```

(Note: the above command will not generate output if the card requires to setup the frontend and/or the demuxer).

You may also wish to simply record the stream with `tzap`, and re-encode it with `ffmpeg` later

```
$ tzap -t <recording duration in seconds> -o foo.ts "<channel name>"
```

3.6 dvbjet

DVB cards receive several simultaneous programs multiplexed. The command-line [dvbjet \(https://github.com/lightful/DVBdirect/\)](https://github.com/lightful/DVBdirect/) standalone tool (has no dependencies) tunes the TV card by selecting the frequency, as with a radio, and saves the *full MPEG-TS stream*. To play or extract a separate program from it (with all its audio, video and subtitle tracks) its companion python script lists the programs and invokes `ffmpeg`.

4 Troubleshooting

4.1 Multiple frontends

Many DVB dongles may register more than 1 frontend. This can be spotted in `dmesg` :

```
[ 9080.196561] usb 1-6: DVB: registering adapter 0 frontend 0 (Realtek RTL2832 (DVB-T))...
[ 9080.196567] dvbdev: dvb_create_media_entity: media entity 'Realtek RTL2832 (DVB-T)' registered.
[ 9080.196626] usb 1-6: DVB: registering adapter 0 frontend 1 (Sony CXD2837ER DVB-T/T2/C demodulator)...
[ 9080.196630] dvbdev: dvb_create_media_entity: media entity 'Sony CXD2837ER DVB-T/T2/C demodulator' registered.
```

They can be listed by doing:

```
$ ls /dev/dvb/adapter0/frontend*
```

Each frontend may have a specific purpose and may only support decoding specific DVB standards. Some software may not have an ability or proper documentation on selecting which frontend to use and will default to `frontend0` , which will cause problems if the broadcast is in a standard that `frontend0` does not support (in above example, if the broadcast is in DVB-T2, but the frontend can only do DVB-T, it will not work)

4.2 Other

If you run into problems, these tools can help debug problems:

- **dvbsnoop** (<https://aur.archlinux.org/packages/dvbsnoop/>)^{AUR} is an advanced tool that can show all the necessary data regarding the bandwidth, signal, frontend, etc.
- `femon -H` shows signal statistics

Retrieved from "<https://wiki.archlinux.org/index.php?title=DVB-T&oldid=841528>"